

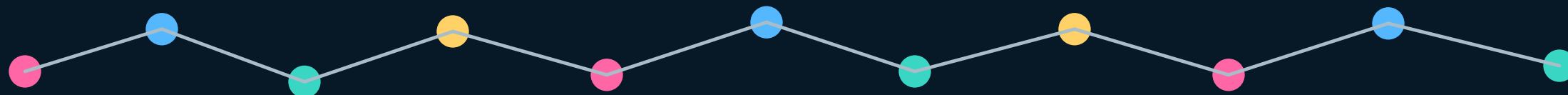
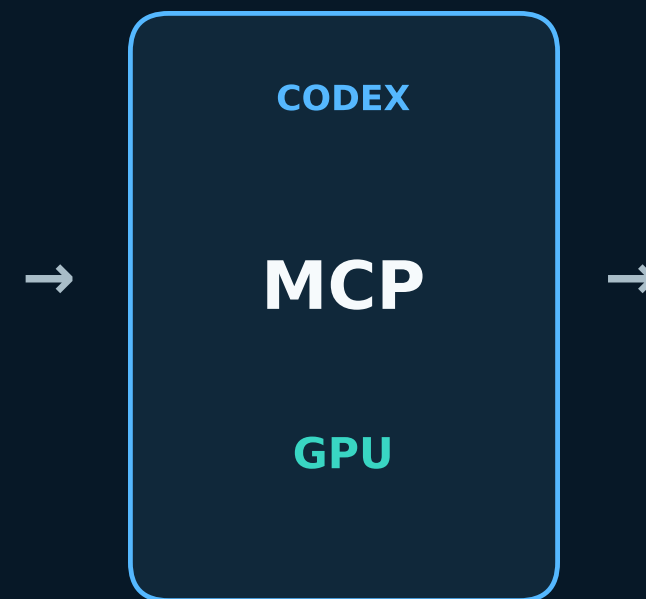
GPU MCP

Let Codex operate the GPU workflow

A short tour for polymer simulators

LESS SSH PLUMBING

MORE REPRODUCIBLE RUNS



MCP in one sentence

MCP gives Codex a typed toolbelt connected to real systems.



MODEL CONTEXT PROTOCOL

Think “USB/API standard for AI tools” — not another model, and not magic autonomy.

The tool schema says what Codex may ask for; the server decides what is actually allowed.

The workflow friction we are removing

TODAY: COMMAND-AND-REMEMBER

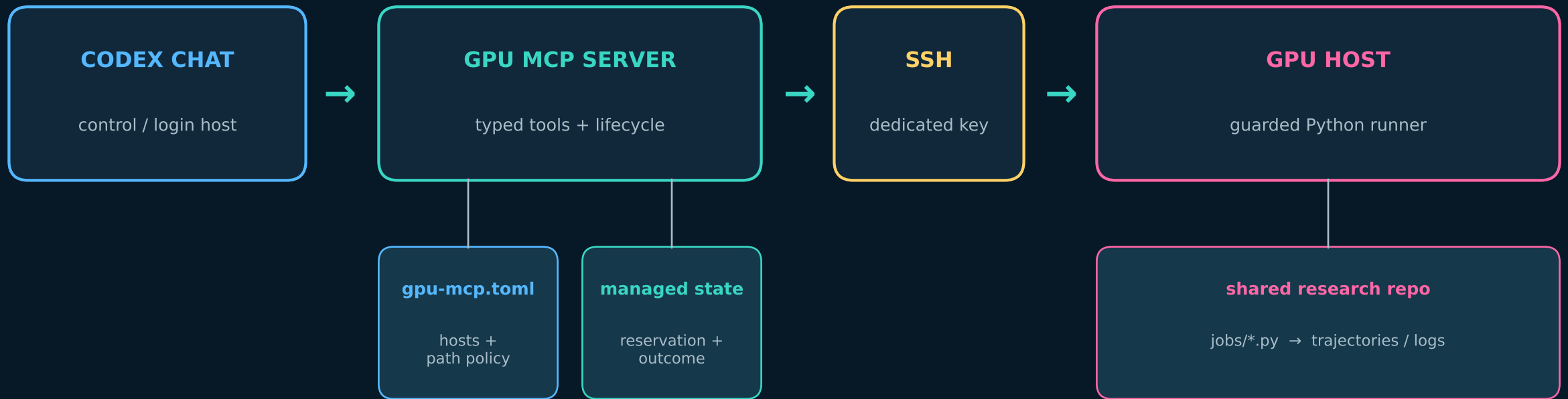
- 1 Check nvidia-smi on several hosts
- 2 Pick a GPU from stale snapshots
- 3 SSH / nohup / redirect logs
- 4 Remember host, PID and output path
- 5 Poll, interpret exit, clean up

WITH GPU MCP: ASK-AND-MANAGE

- 1 Describe the simulation goal
- 2 Codex checks availability
- 3 MCP reserves + launches safely
- 4 One job_id tracks the lifecycle
- 5 Status reports outcome + outputs

POLYMER-SIMULATION PAYOFF Less run shepherding; clearer evidence about what ran, where, and whether it finished.

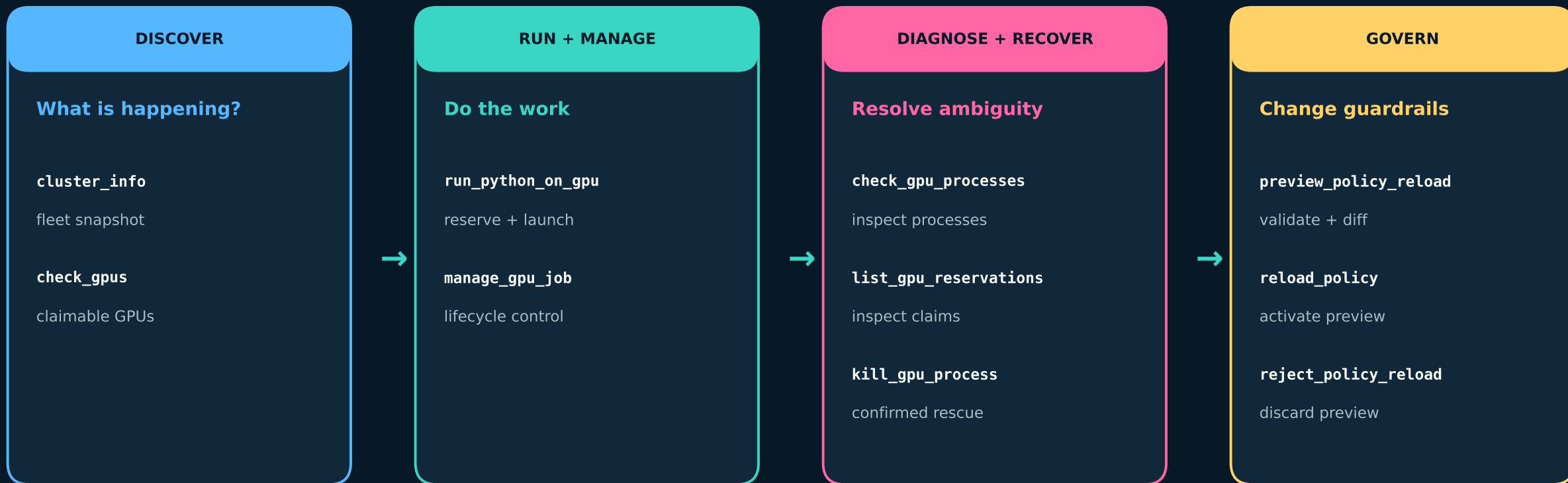
What happens after you ask Codex



The MCP runs beside Codex — middleware under our control, not a hosted cloud service.

The MCP is an ensemble of tools

No single tool is “the MCP” — the value comes from how the tools compose.



observe → claim → run → verify → intervene → evolve policy

The everyday run loop: three tools



The `job_id` is the thread that connects launch, status, recovery, and final outcome.

The supporting tools: see, recover, govern

SEE THE FLEET

`cluster_info`

Reachability, GPU counts, average utilization, load.

`check_gpu_processes`

Host / GPU / PID / user / command and memory for compute processes.

RECOVER SAFELY

`list_gpu_reservations`

Active claims for me or everyone; optional bounded refresh.

`kill_gpu_process`

Two-step, fingerprint-confirmed rescue for one owned PID.

GOVERN POLICY

`preview_policy_reload`

Validate the changed policy, show the diff, issue a one-time token.

`reload_policy`

Activate that exact approved hash.

`reject_policy_reload`

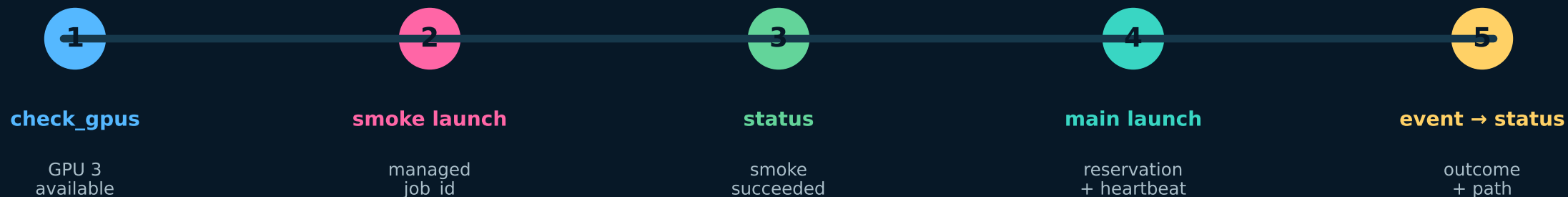
Discard the token; change nothing.

These tools make the normal run loop observable, recoverable, and policy-bounded.

A polymer workflow, end to end

YOU

“Smoke-test the bead-spring equilibration on an available GPU.
If it passes, launch production and tell me where outputs land.”



Codex owns the plumbing — not the polymer physics.

We still choose the model, ensemble, equilibration criteria, and what counts as credible.

Bounded autonomy: the safety model

1

REPO POLICY

approved hosts and script / write roots

2

GUARDED PYTHON

no arbitrary remote shell; bounded writes

3

GPU RESERVATION

atomic claim prevents cooperative double-booking

4

MANAGED JOB

heartbeat, outcome record, explicit job_id

5

FAIL CLOSED

unknown process state stays reserved

WHAT IT IS

**A cooperative guardrail
for trusted lab workflows**

**A reproducible tool path
Codex can reason about**

WHAT IT IS NOT

**Not Slurm or a queue
Not a hostile-code sandbox
Not a validator of force fields
or equilibration**

Three things to remember

1

MCP is the bridge

Codex gets typed tools;
the server keeps authority.

2

The tools work as a system

Discover → run → manage
→ recover → govern.

3

We keep scientific judgment

The tool validates mechanics,
not the physics.

GOOD FIRST USE

Ask Codex to run one bounded smoke job and report job_id, outcome, and output path.

Questions?